



College of Basic and Applied Sciences
School of Engineering Sciences · Department of Computer Engineering

CPEN 208 — Project 2

API / Web Service implementing the Project 1 functions

Name: David Kwame Odoi-Anim

Student ID: 22312110

Course: CPEN 208 — Introduction to Software Engineering

Semester: Second Semester, 2025/2026

Lecturer: Mr. Asiamah

Spring Boot 3.5

Java 21

Spring JDBC

JWT

BCrypt

GitHub: <https://github.com/doxaodoi/project-1-2>

1. Overview

Project 2 is an **API / web service** that implements the functions created in Project 1 and exposes them over HTTP as JSON. It is a Spring Boot application (Java 21) that connects to the same `coe_dept` PostgreSQL database and runs the Project 1 SQL — including the `finance.get_outstanding_fees()` function — through Spring's `JdbcTemplate`. The Project 1 front end consumes this service.

```
Browser → Next.js 14 front end (:3000) → Spring Boot API (:8080) → PostgreSQL
                                           coe_dept
```

1.1 Why a separate service

Keeping the API separate from the front end means the database logic lives in one place, behind a documented contract. The browser never holds database credentials or raw SQL; it only sees JSON responses and a signed token.

2. Technology & Structure

Concern	Choice
Language / runtime	Java 21
Framework	Spring Boot 3.5 (Spring Web)
Database access	Spring JDBC (<code>JdbcTemplate</code>) — runs the Project 1 SQL directly
Password hashing	BCrypt (<code>spring-security-crypto</code>)
Authentication	Stateless JWT (HS256), verified by a servlet filter
Build	Maven

The code is layered: `controller` → `service` → `repository` (`JdbcTemplate`), with `dto`, `security` (JWT util + filter) and `config` (CORS, beans) packages.

3. API Endpoints

Method & path	Auth	Description
POST /api/auth/register	public	Create a student + login account, return a JWT
POST /api/auth/login	public	Verify credentials, return a JWT
GET /api/me	JWT	Current student profile + fee summary
GET /api/students/{id}	JWT	Student personal information
GET /api/students/{id}/enrollments	JWT	Enrolled courses + assigned lecturer
GET /api/students/{id}/payments	JWT	Payment history
GET /api/students/{id}/outstanding	JWT	Billed / paid / outstanding for the student
GET /api/fees/outstanding	JWT	The Project 1 JSON function for all students

CPEN 208 Project 2 — CoE Department API

The API is running. Endpoints:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/me (auth)
- GET /api/students/{id} (auth)
- GET /api/students/{id}/enrollments (auth)
- GET /api/students/{id}/payments (auth)
- GET /api/students/{id}/outstanding (auth)
- GET /api/fees/outstanding (auth) — implements the Project 1 JSON function

The API landing page at <http://localhost:8080/> listing the available endpoints.

4. Authentication & Security

- **Passwords** are hashed with BCrypt. The seeded accounts and any newly-registered account use the same encoder, so both verify identically.

- **Login/registration** return a signed **JWT** (HS256). The token carries the student id and email and an expiry.
- A **servlet filter** (`JwtAuthFilter`) protects every `/api/**` route except `/api/auth/**`. A missing or invalid token yields `401`.
- **Ownership check:** a student may only read their own records — the token's subject must match the `{id}` in the path, otherwise `403`.

1. POST `/api/auth/login {email, password}`
2. API verifies BCrypt hash → issues JWT
3. Front end stores JWT in an HttpOnly cookie
4. Each request sends `Authorization: Bearer <token>`
5. `JwtAuthFilter` validates the signature & expiry before the controller runs

5. Implementing the Project 1 Function

The headline endpoint, `GET /api/fees/outstanding`, executes the Project 1 database function and streams its JSON straight to the client:

```
// FeesRepository.java
public String getAllOutstandingJson() {
    return jdbc.queryForObject(
        "SELECT finance.get_outstanding_fees()::text", String.class);
}
```

```
// FeesController.java
@GetMapping("/outstanding")
public ResponseEntity<String> outstanding() {
    return ResponseEntity.ok()
        .contentType(MediaType.APPLICATION_JSON)
        .body(fees.getAllOutstandingJson()); // 70-element JSON array
}
```

5.1 Live response (excerpt)

```
[
  { "student_id": 22384451, "full_name": "Abu Neaquittae Golda",
    "total_billed": 12300.00, "total_paid": 12300.00, "outstanding": 0.00 },
  { "student_id": 22312110, "full_name": "David Kwame Odoi-Anim",
    "total_billed": 12300.00, "total_paid": 9000.00, "outstanding": 3300.00 }
  /* ... 70 students total ... */
]
```

5.2 The dashboard is powered by this API

The screenshot below is the Project 1 dashboard; every value on it — profile, fee summary, courses and payments — is fetched from this service (`/api/me`, `/api/students/{id}/enrollments`, `/api/students/{id}/payments`).

Signed in as
David Kwame Odoi-Anim
Student ID 22312110
Email 22312110@st.ug.edu.gh
Programme BSc Computer Engineering
Level 200

Total billed
GH¢12,300.00

Total paid
GH¢9,000.00

Outstanding
GH¢3,300.00

Enrolled Courses

Code	Title	Credits	Lecturer	Grade
CPEN 202	Object Oriented Programming	3	Dr. Kwame Mensah	In progress
CPEN 204	Data Structures and Algorithms	3	Dr. Ama Boateng	In progress
CPEN 206	Linear Circuits	3	Dr. Yaw Owusu	In progress
CPEN 208	Introduction to Software Engineering	3	Prof. Elsie Adjei	In progress
CPEN 210	Digital Logic Design	3	Dr. Efua Danso	In progress
MATH 223	Multivariable Calculus	3	Mr. Kofi Antwi	In progress

Payment History

Date	Amount	Method	Reference
2025-06-12	GH¢6,000.00	BANK	PAY-22312110-1
2025-07-14	GH¢3,000.00	MOMO	PAY-22312110-2

Front-end dashboard rendering data served by the Spring Boot API.

6. How to Run

1. Ensure the `coe_dept` database exists (Project 1 scripts or the provided backup).
2. `cd "PROJECT 2/api"`
3. Provide the database password, e.g. `set DB_PASSWORD=your_postgres_password`.
4. `mvn spring-boot:run` — the API starts on `http://localhost:8080`.
5. Verify: open `http://localhost:8080/`, or `POST /api/auth/login` then `GET /api/fees/outstanding`.

Deliverables in this folder: the Spring Boot source is in `PROJECT 2/api`; the database scripts and backup are in `PROJECT 2/database`; all source code is on GitHub at <https://github.com/doxaodoi/project-1-2>.